2026 VLSI Design and Implementation

LAB4 FIFO with Latch

Objective

In this assignment, students will explore the differences between edge-triggered flip-flops and level-sensitive latches, practicing the use of D-latches as primary storage elements to ultimately implement a **Synchronous First-In-First-Out (FIFO) buffer based on a latch-array architecture**.
Due Date: 2026/5/29 12:00 p.m.

Lab4 Download

---

1. Download the provided environment using the following command.
   tar xvf /home/TA00/VLSI_2026/Lab4.tar
2. The extracted LAB directory contains:
   - `00_TESTBED`: Contains the testbench and pattern files .
   - `01_RTL`: Your workspace for the Verilog design (`FIFO.v`).
   - `02_SYN`: Synthesis scripts and working directory.
   - `03_GATE`: Gate-level netlist and simulation files.

Background Knowledge

---

1. Latch
   - **Definition:**
     A latch is a **level-sensitive** storage element. It becomes "transparent" (input passes directly to output) when its enable signal is active, and holds its current state when the enable signal is inactive.

   - **Advantages:**
     Compared to standard edge-triggered D Flip-Flops (DFFs), latches require about half the number of transistors. This translates to a significantly **smaller silicon area** and **lower leakage power**, making them highly desirable for custom memory arrays (e.g., SRAM) and low-power designs.

   - **Disadvantages & Design Challenges:**
     Because latches are level-sensitive, they are highly vulnerable to **glitches** on the enable pin, which can instantly corrupt stored data.

Furthermore, latch-based designs introduce complex timing behaviors like **Time Borrowing**, which makes Static Timing Analysis (STA) and hold-time closure much more difficult than in DFF-based designs.

2. FIFO (More detail in **Lab4_FIFO.pdf**)
    - **Definition:**
      A FIFO is a data buffer commonly used in digital systems to manage data flow between different processing modules. As the name implies, data is read out in the exact order it was written.

    - **Pointer Management:**
      Instead of physically shifting data through a chain of registers, a standard FIFO utilizes a **Write Pointer** and a **Read Pointer** to track and address specific locations within a stationary internal memory array.

    - **Status Flags & Protection:**
      A robust FIFO must accurately generate **Full** and **Empty** status flags to indicate its current capacity. It must also implement hardware-level protections to prevent **Overflow** (ignoring write commands when the buffer is full) and **Underflow** (ignoring read commands when the buffer is empty).

Design Description

In this lab, you are required to design a **Synchronous Dual-Port FIFO** that supports simultaneous read and write operations. Your design must strictly comply with the following architectural and behavioral specifications:

1. Storage Element Constraint (Latch-Based Array)

- **High-Level Sensitive Latches:**
  The core memory array of the FIFO **MUST** be implemented using D-latches rather than standard D-flip-flops (DFFs).

- These latches must be **high-level sensitive**, meaning they are transparent (enabled to capture data) when `clk = 1` and opaque (holding data) when `clk = 0`.
  ***Note**:* The clock can operate with other enable signals to control the switching action of the latch.
  ***Note**:* You are allowed to use standard edge-triggered DFFs for the control logic (e.g., Write/Read pointers and state flags).

- **Depth & Width:**
  The FIFO module must be **fully parameterized**. Hardcoding the dimensions of the memory array, pointers, or data buses is strictly prohibited.

  `DATA_WIDTH`: Defines the number of bits per data entry (i.e., the bus width for the `w_data` and `r_data` ports).
  Note
  ***Note**:* TA will test when **`DATA_WIDTH`** is set to 8 and **16**.

  `FIFO_DEPTH`: Defines the maximum number of data entries the FIFO can store.
  ***Note**:* TA will test when **`FIFO_DEPTH`** is set to 16 and **32**.

2. Status Flags (Empty & Full)

The FIFO must output accurate status flags to indicate its current capacity.

- **Empty (`empty`):**
  - Asserted (`empty = 1`) immediately after an asynchronous reset (`rst_n = 0`).
  - Asserted (`empty = 1`) whenever there is no valid data stored in the FIFO.
  - De-asserted (`empty = 0`) when there is at least one valid data entry.
- **Full (`full`):**
  - Asserted (`full = 1`) when the FIFO reaches its maximum capacity (`FIFO_DEPTH`).
  - De-asserted (`full = 0`) when the FIFO has available space for new incoming data.

3. Operation Modes (Read & Write)

The FIFO must correctly handle individual and simultaneous read/write operations while implementing hardware-level protections against invalid commands.

- **Read Only (`r_en = 1`, `w_en = 0`):**
  - **Valid Read:**
    If `empty = 0`, the read pointer increments at the positive edge of the clock.
    (Assuming **FWFT (First-Word Fall-Through)** architecture, the oldest data should already be present at the `r_data` output).

- **Underflow Protection:**
  If `empty = 1`, the read operation is considered **invalid**. The hardware must ignore the `r_en` signal. The read pointer MUST NOT increment, and the `empty` flag must remain `1`.

- **Write Only (`w_en = 1`, `r_en = 0`):**
  - **Valid Write:**
    If `full = 0`, the input data (`w_data`) is written into the memory array, and the write pointer increments at the positive edge of the clock.

  - **Overflow Protection:**
    If `full = 1`, the write operation is considered **invalid**. The hardware must ignore the `w_en` signal. The write pointer MUST NOT increment, and existing data inside the FIFO MUST NOT be overwritten or corrupted.

- **Simultaneous Read and Write (`r_en = 1`, `w_en = 1`):**
  - **Normal Case (Not Full, Not Empty):**
    Both read and write operations execute successfully. Both pointers increment. The `full` and `empty` flags remain unchanged since the total number of stored items is conserved.

  - **Boundary Case 1 - Empty (`empty = 1`):**
    The write operation is valid, but the read operation is **invalid** (blocked by underflow protection). In the next cycle, `empty` should transition to `0`, and the newly written data becomes available.

  - **Boundary Case 2 - Full (`full = 1`):**
    The read operation is valid, but the write operation is **invalid** (blocked by overflow protection). In the next cycle, `full` should transition to `0`, freeing up space.

Signal Description

Parameter	Type	Description
`FIFO_DEPTH`	int	Defines the maximum number of data entries the FIFO can store.
`DATA_WIDTH`	int	Defines the bit-width of each data entry.

Table. 1. Parameter list

Signal	In / Out	Bit	Description
`clk`	In	1	System Clock.
`rst_n`	In	1	Asynchronous active-low reset.
`w_en`	In	1	Write enable signal.
`w_data`	In	`DATA_WIDTH`	Write data bus.
`r_en`	In	1	Read enable signal.
`r_data`	Out	`DATA_WIDTH`	Read data bus.
`full`	Out	1	Full status flag. High when the FIFO reaches its maximum capacity.
`empty`	Out	1	Empty status flag. High when there is no valid data in the FIFO.

Table. 2. I/O port list

Design Constraints

01_RTL

1. Top module name: `FIFO.v`
2. The reset signal (`rst_n`) would be given only once at the beginning of the simulation. The empty signal should be asserted (set to 1) immediately after the reset signal is asserted.
3. Your design must pass the pattern.

02_SYN

1. The synthesis result **must contain latch**.
   Your syn.log will contain something similar to the image below.

```
==============================================================================
|      Register Name      | Type  | Width | Bus | MB | AR | AS | SR | SS | ST |
==============================================================================
| sram_row[0].cell_q_reg  | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[1].cell_q_reg  | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[2].cell_q_reg  | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[3].cell_q_reg  | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[4].cell_q_reg  | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[5].cell_q_reg  | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[6].cell_q_reg  | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[7].cell_q_reg  | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[8].cell_q_reg  | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[9].cell_q_reg  | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[10].cell_q_reg | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[11].cell_q_reg | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[12].cell_q_reg | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[13].cell_q_reg | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[14].cell_q_reg | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
| sram_row[15].cell_q_reg | Latch |  16   |  Y  | N  | N  | N  | -  | -  | -  |
==============================================================================
```

Fig. 1. Latch in syn.log

2. The cycle time is **fixed to 20ns**.
3. After synthesis, the design must meet timing requirements. The timing report is valid only if the final slack reported in `FIFO.timing` is **MET**.
   It is acceptable if the timing report contains Timing Borrowing Information.

```
Startpoint: w_data[0] (input port clocked by clk)
Endpoint: u_latch_mem/sram_row_0__cell_q_reg_0_
          (positive level-sensitive latch clocked by clk)
Path Group: clk
Path Type: max

Point                                                   Incr       Path
---------------------------------------------------------------------------
clock clk (rise edge)                                   0.00       0.00
clock network delay (ideal)                             0.00       0.00
input external delay                                    6.00       6.00 f
w_data[0] (in)                                          0.00       6.00 f
u_latch_mem/w_data[0] (latch_array_mem_DATA_WIDTH16_FIFO_DEPTH16_ADDR_WIDTH4)
                                                        0.00       6.00 f
u_latch_mem/U3/Y (INVX16)                               0.08       6.08 r
u_latch_mem/U19/Y (INVX3)                               0.06       6.14 f
u_latch_mem/sram_row_0__cell_q_reg_0_/D (TLATX1)        0.00       6.14 f
data arrival time                                                  6.14

clock clk (rise edge)                                   0.00       0.00
clock network delay (ideal)                             0.00       0.00
clock uncertainty                                      -0.10      -0.10
u_latch_mem/sram_row_0__cell_q_reg_0_/G (TLATX1)        0.00      -0.10 r
time borrowed from endpoint                             6.24       6.14
data required time                                                 6.14
---------------------------------------------------------------------------
data required time                                                 6.14
data arrival time                                                 -6.14
---------------------------------------------------------------------------
slack (MET)                                                        0.00

Time Borrowing Information
-----------------------------------------------------------
clk nominal pulse width                                10.00
library setup time                                     -0.11
-----------------------------------------------------------
max time borrow                                         9.89
-----------------------------------------------------------
actual time borrow                                      6.24
clock uncertainty                                      -0.10
-----------------------------------------------------------
time given to startpoint                                6.14
-----------------------------------------------------------
```

Fig. . Timing report with no timing violation

4. The synthesis result cannot contain any **Error**.

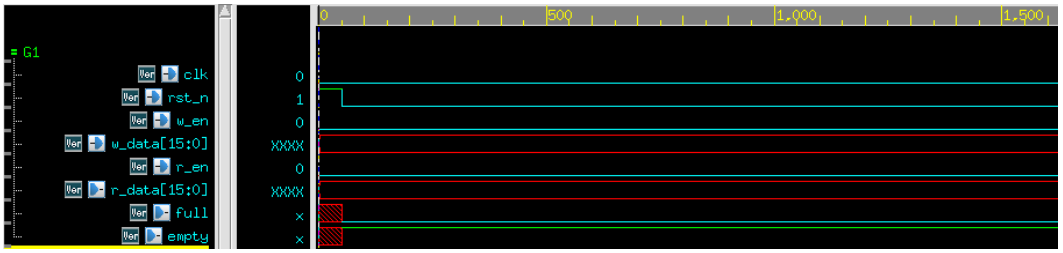
03_GATE

1. All requirements in 01_RTL must be met after including timing checks in Gate-Level simulation.

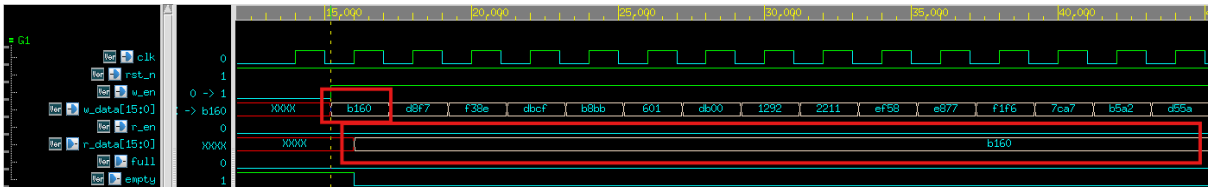
Waveform Examples

---

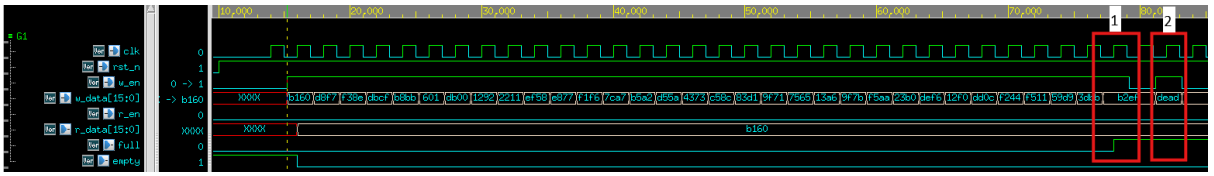
1. After reset, `empty` must be asserted to 1 and `full` must be 0.



2. `r_data` should be the next data available for reading, and it must be instantly available at the output immediately upon being written into the FIFO.

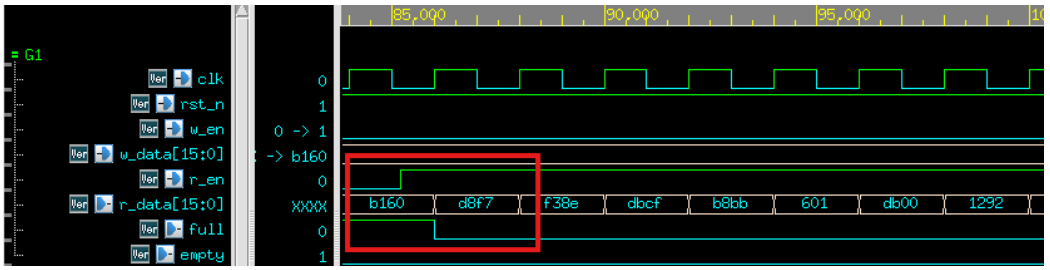


3. After the FIFO is full, `full` should be high in the same cycle, and the new data should not be written into FIFO.



    1. `full` should be high (at posedge clk) after the final data is written into FIFO.
    2. When the FIFO is full, incoming data should not be written into FIFO.

4. The Pattern will immediately check the `r_data` as soon as `r_en` is asserted high. Subsequently, at the next positive edge of clk, `r_data` should update to the next oldest valid data entry.



Grading Policy

---

1. **Function Validity: 60%**
   Only passing 01_RTL simulation - 30%
   Passing RTL and Gate-level simulation without timing violations - 30%
2. **Performance: 30%**
   Performance Score = `Power * Area`
   ※The Performance Score calculation will be based on `DATA_WIDTH` = 16 and `FIFO_DEPTH` = 32.
   ※`Power` is “Total Power” in your power report.
   ※`Area` is “Total cell area” in your area report.
3. **Report: 10%**
   The Report must include the following items:
   a. Explain how you control the latches in your design.
      Specifically, detail your strategy for determining when the latches are enabled and disabled to safely capture data while preventing glitches or unintended overwrites.
   b. The area information for all four combinations of `DATA_WIDTH` (8 and 16) and `FIFO_DEPTH` (16 and 32).
   c. Analyze the advantages and disadvantages of using a latch as the core storage element for the FIFO design in this lab.
4. **Due date: 2026/5/29 12:00 p.m.**
   Please submit the following zip files to the E3 system before 12:00 at noon in May. 29
   - vlsixx.zip (ex. vlsi00.zip)
     - vlsixx (folder)
       - FIFO.v
       - vlsixx_report.pdf
5. If uploaded files violate the naming rule, you will get 5 deduct points.

Notes

- Submission
  - 1st Demo
    Submit and pass all test cases before 12:00 noon on May 29, 2026.
    The score will be calculated according to the grading policy.

  - 2nd Demo
    If you do not pass all test cases or fail to submit (in the 1st Demo), and submit and pass all test cases before 12:00 noon on June 5, 2026, you will receive the Function score(70%).

- Reference Commands
  - 01_RTL/ (RTL simulation) ./01_run_vcs_rtl
  - 02_SYN/ (Synthesis) ./01_run_dc_shell
  - 03_GATE / (Gate-level simulation) ./01_run_vcs_gate

- Power

```
                 Internal         Switching           Leakage            Total
Power Group      Power            Power               Power              Power   (   %    )  Attrs
--------------------------------------------------------------------------------------------------
io_pad             0.0000           0.0000              0.0000             0.0000  (   0.00%)
memory             0.0000           0.0000              0.0000             0.0000  (   0.00%)
black_box          0.0000           0.0000              0.0000             0.0000  (   0.00%)
clock_network   3.4559e-02       6.6880e-04          2.1479e+05         3.5443e-02  (  48.90%)  i
register        5.4230e-03       4.9681e-04          1.0256e+07         1.6176e-02  (  22.32%)
sequential         0.0000           0.0000              0.0000             0.0000  (   0.00%)
combinational   6.9396e-03       1.0772e-02          3.1559e+06         2.0868e-02  (  28.79%)
--------------------------------------------------------------------------------------------------
Total           4.6922e-02 mW    1.1938e-02 mW       1.3627e+07 pW      7.2487e-02 mW
```

- How to modify FIFO_DEPTH & DATA_WIDTH
  1. In 00_TESTBED/TESTBED.v

```
29    module TESTBED;
30
31    parameter DATA_WIDTH  = 16;
32    parameter FIFO_DEPTH  = 32;
```

  2. Before synthesizing the RTL, ensure the parameters in FIFO.v are synchronized with the TESTBED.v.
     Failure to do so will result in a synthesized netlist with inconsistent or incorrect dimensions.

```
module FIFO #(
    parameter DATA_WIDTH = 16,
    parameter FIFO_DEPTH = 32
)(
    input                   clk,
    input                   rst_n,
    input                   w_en,
    input  [DATA_WIDTH-1:0] w_data,
    input                   r_en,
    output [DATA_WIDTH-1:0] r_data,
    output                  full,
    output                  empty
);
```